



目标识别

- 目标识别面临的挑战
 - 挑战 1: 目标种类多
 - 挑战 2: 视点变化
 - 挑战 3: 亮度变化
 - 挑战 4: 遮挡
 - 挑战 5: 尺度
 - 挑战 6: 形变
 - 挑战 7: 复杂背景
 - 挑战 8: 类内变化大



人脸检测

- 分类器
 - AdaBoost (Adaptive Boosting) 分类器
 - Boosting: 是一种提高任意给定学习算法准确度的方法。这种方法依次顺序地加入多个分量分类器(弱分类器即可), 其中每个分量分类器的训练集都选自己有的其他各个分类器所给出的“最富信息(most informative)”的样本点组成, 而最终的判决结果则是根据这些分量分类器的结果共同决定。
 - “三个臭皮匠, 赛过诸葛亮”

49



人脸检测

- 分类器
 - AdaBoost (Adaptive Boosting) 分类器
 - 允许设计者不断地加入新的弱分类器, 直到达到某个预定的足够小的错误率。
 - 每个训练样本都被赋予一个权重, 表明它被某个分量分类器选中的概率。如果每个样本点已经被准确地分类, 那么在构造下一个训练集的时候, 它被选中的概率就降低; 相反, 如果某个样本点没有被正确地分类, 那么它的权重得到提高。

- 分类器

- AdaBoost (Adaptive Boosting) 分类器

Given: $(x_1, y_1), \dots, (x_m, y_m)$ where $x_i \in X, y_i \in Y = \{-1, +1\}$

Initialize $D_1(i) = 1/m$.

For $t = 1, \dots, T$:

- Train weak learner using distribution D_t .
- Get weak hypothesis $h_t : X \rightarrow \{-1, +1\}$ with error

$$\epsilon_t = \Pr_{i \sim D_t} [h_t(x_i) \neq y_i].$$

- Choose $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$.

- Update:

$$\begin{aligned} D_{t+1}(i) &= \frac{D_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & \text{if } h_t(x_i) = y_i \\ e^{\alpha_t} & \text{if } h_t(x_i) \neq y_i \end{cases} \\ &= \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t} \end{aligned}$$

where Z_t is a normalization factor (chosen so that D_{t+1} will be a distribution).

Output the final hypothesis:

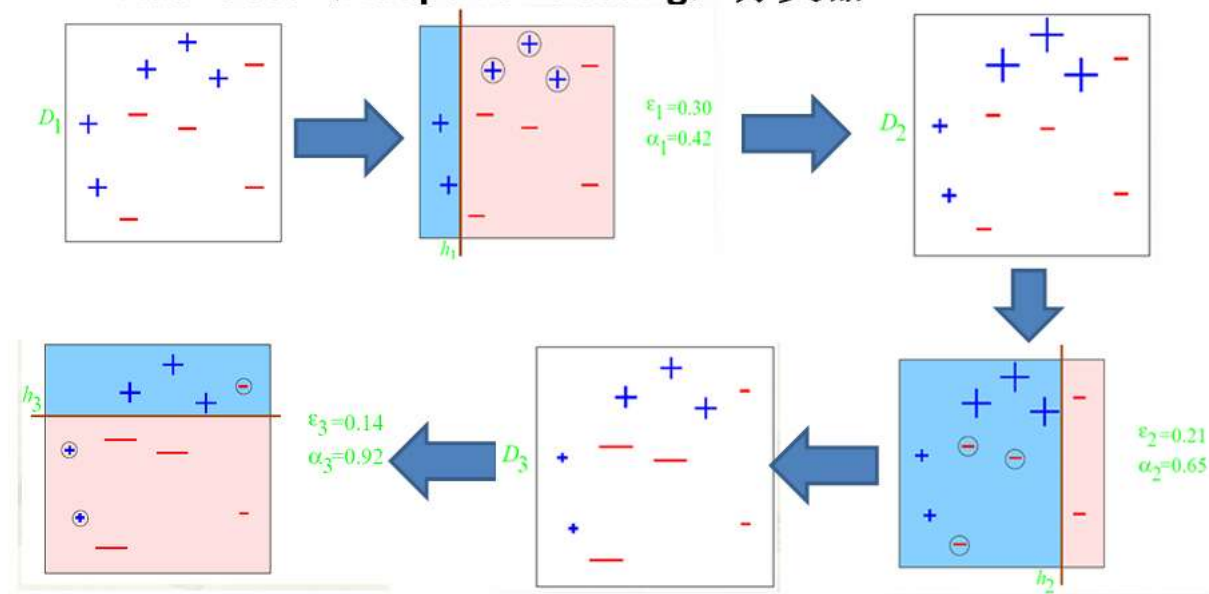
$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$



人脸检测

- 分类器

- AdaBoost (Adaptive Boosting) 分类器



2. 核心大循环 (For $t = 1, \dots, T$)

算法会进行 T 轮迭代，每一轮都会训练出一个“弱分类器”(也就是之前提到的决策桩，只看一个特征)。

- 计算错误率 (ϵ_t):

分类器 h_t 跑完后，看看它分错了哪些样本。错误率 ϵ_t 就是所有被分错的样本目前的权重之和。

- 计算分类器的话语权 (α_t):

这是 AdaBoost 的神来之笔。公式是：

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

意思很简单：如果这个弱分类器错误率很低 (ϵ_t 很小)，那么它计算出的 α_t 就会很大，意味着在最终投票时，它拥有很高的“话语权”。反之，如果它也就是乱猜的水平 (ϵ_t 接近 0.5)，它的 α_t 就会接近 0。

- 更新样本权重 (D_{t+1}):

这是 “Adaptive (自适应)” 的真正体现。

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & \text{如果分对了} \\ e^{\alpha_t} & \text{如果分错了} \end{cases}$$

- 如果样本被分错了，说明它是个“硬骨头”。我们就给它乘上 e^{α_t} (一个大于 1 的数)，放大它的权重。
- 如果样本被分对了，说明它很简单。我们就给它乘上 $e^{-\alpha_t}$ (一个小于 1 的数)，缩小它的权重。
- 最后除以 Z_t (归一化因子)，保证所有样本的新权重加起来还是等于 1。

下一轮训练新的弱分类器时，算法就会被迫去死磕那些上一轮被放大了权重的“错题”。

• AdaBoost（Adaptive Boosting）分类器

– 弱分类器

- 决策桩(stumps)

$$h(x, f, p, \theta) = \begin{cases} 1 & \text{if } pf(x) > p\theta \\ -1 & \text{otherwise} \end{cases}$$

- 每一个弱分类器由一个特征组成

– 正负样本不平衡时：

- 初始化权重： $D_{1,i} = \frac{1}{2m}, \frac{1}{2n}$ ， m, n 分别为正样本、负样本数量

- 初始化权重公式拆解：

为了杜绝这种“作弊”，算法在第一轮训练开始前，强行给两类样本分配了初始的“重要程度（权重） D_1 ”：

- 正样本（人脸）的初始权重： $D_{1,i} = \frac{1}{2m}$
- 负样本（背景）的初始权重： $D_{1,i} = \frac{1}{2n}$

这个公式的精妙之处在于：

把所有 m 个人脸的权重加起来： $m \times \frac{1}{2m} = \frac{1}{2}$ （也就是 50% 的重要性）。

把所有 n 个背景的权重加起来： $n \times \frac{1}{2n} = \frac{1}{2}$ （也是 50% 的重要性）。

总结下半部分：通过这个公式，无论背景样本的数量是人脸的十倍、百倍还是万倍，在起跑线上，“全部人脸”的总地位和“全部背景”的总地位被强行按成了 1:1 五五开。如果模型还敢无脑猜全都是背景，它立刻就会因为丢掉了人脸那 50% 的权重分数而不及格。这就逼迫模型必须老老实实去学习真正能区分两者的特征。

- 核心数学公式拆解：

$$h(x, f, p, \theta) = \begin{cases} 1 & \text{if } pf(x) > p\theta \\ -1 & \text{otherwise} \end{cases}$$

这个公式就是这个单细胞评委的“工作手册”：

- x : 当前正在被检查的图像小窗口。
- f : 这个评委被分配到的那**唯一**一个特征（比如“黑白黑”的线条特征）。
- $f(x)$: 在这张图上，计算出这个特征的**具体数值**。
- θ (**阈值**): 评委心里的“及格线”。
- p (**极性 / Parity**): 这是一个非常巧妙的数学技巧，取值只能是 1 或 -1 。它的存在是为了控制“大于及格线算人脸”还是“小于及格线算人脸”。
- 当 $p = 1$ 时，不等式是 $f(x) > \theta$ 。代表：特征值**大于**阈值时，我才认为是人脸 (1)。
- 当 $p = -1$ 时，不等式是 $-f(x) > -\theta$ ，两边同乘 -1 也就是 $f(x) < \theta$ 。代表：特征值**小于**阈值时，我才认为是人脸 (1)。

总结上半部分：一个弱分类器 h ，拿着它专属的特征 f ，在图片 x 上算出一个分，根据方向 p 看看有没有通过及格线 θ 。通过了就举牌说“是人脸 (1)”，没通过就举牌说“不是 (-1)”。



人脸检测

- 层级式(Cascade)分类器
 - 对于一幅图像，只有极少量扫描窗中存在人脸。也就是说大多数扫描窗中都是非人脸。
 - 使用少量的特征组成的分类器排除掉大部分非人脸的区域。
 - “先易后难”



人脸检测

• 层级式(Cascade)分类器

- User selects values for f , the maximum acceptable false positive rate per layer and d , the minimum acceptable detection rate per layer.
- User selects target overall false positive rate, F_{target} .
- P = set of positive examples
- N = set of negative examples
- $F_0 = 1.0$; $D_0 = 1.0$
- $i = 0$
- while $F_i > F_{target}$
 - $i \leftarrow i + 1$
 - $n_i = 0$; $F_i = F_{i-1}$
 - while $F_i > f \times F_{i-1}$
 - * $n_i \leftarrow n_i + 1$
 - * Use P and N to train a classifier with n_i features using AdaBoost
 - * Evaluate current cascaded classifier on validation set to determine F_i and D_i .
 - * Decrease threshold for the i th classifier until the current cascaded classifier has a detection rate of at least $d \times D_{i-1}$ (this also affects F_i)
- $N \leftarrow \emptyset$
- If $F_i > F_{target}$ then evaluate the current cascaded detector on the set of non-face images and put any false detections into the set N

第一步：设定“通关 KPI”(参数初始化)

在建立任何关卡之前，你需要先设定好游戏规则：

- **单关保底生存率 (d)**：每一关必须保证多高的人脸存活率？（比如 $d = 99.5\%$ ，原则是：宁可错放背景，绝不漏杀人脸）。
- **单关淘汰率 (f)**：每一关必须淘汰掉多少比例的背景？（比如 $f = 50\%$ ，每一关都要把剩下的非人脸刷掉一半）。
- **终极防线 (F_{target})**：整个游戏全部通关后，允许的整体最终误报率是多少？（比如 10^{-6} ，要求百万挑一的极严苛标准）。
- **准备初始题库**：准备好正样本库（全是人脸 P ）和初始的负样本库（全是背景 N ）。

第二步：打造当前关卡（内层循环 —— 训练单层分类器）

假设我们正在建造第 i 关。此时我们的任务是：用尽量少的特征，达到这一关的 KPI。

1. **从零开始：** 这一关刚建立时，没有任何特征（包含的弱分类器数量 $n_i = 0$ ）。
2. **增加考题（加特征）：** 往这一关里增加 1 个特征，让 AdaBoost 拿着当前的 P 和 N 题库进行训练。
3. **强行及格（降阈值）：** AdaBoost 训练完后，为了保证人脸不被淘汰，算法会强行拉低这关的及格线（分类阈值），直到人脸的通过率达到你设定的 d （99.5%）。
4. **检查淘汰率：** 及格线拉低后，肯定会有更多的背景“蒙混过关”。此时去看看背景误报率是不是超过了 f （50%）。
 - 如果没达标（混过去的背景大于 50%）： 关卡太简单了。回到第 2 步，再加 1 个特征，继续让 AdaBoost 练。
 - 如果达标了（混过去的背景小于 50%）： 恭喜，第 i 关建设完成！锁定这关的特征集和及格线。

这是一个非常核心的问题。要理解“阈值”和“指标”，我们需要看看 AdaBoost 在这一关（这一层）最后交卷时，到底是怎么打分的。

1. 这里的“阈值（Threshold）”到底是什么？

在刚才的“闯关游戏”中，每一关（层）实际上是一个由 AdaBoost 训练出来的强分类器。这个强分类器是由若干个（比如 10 个）看单一特征的“弱分类器”投票组合而成的。

当一个图像窗口进入这一关时，它的最终得分计算公式是：

总得分 = 评委1的话语权 × 评委1的投票 + ... + 评委N的话语权 × 评委N的投票
计算出这个“总得分”后，系统需要一条及格线来拍板：

- 如果 总得分 > 及格线（阈值） → 判断为是人脸，放行到下一关。
- 如果 总得分 < 及格线（阈值） → 判断为是背景，直接淘汰。

在标准的 AdaBoost 中，这条及格线默认是 0。但在级联分类器的训练中，算法会强行向左移动（拉低）这条及格线，可能把它变成 -0.5 甚至 -1.2。

第三步：搜集“硬骨头”(困难样本挖掘 —— Hard Negative Mining)

这是级联模型越往后越强悍的核心秘密。

第 i 关建好后，原本题库里那些容易识别的背景（比如纯色天空、白墙）早就被淘汰光了。为了训练下一关（第 $i + 1$ 关），我们需要找更难的题：

1. **清空原题库：** 把当前的负样本池 N 直接清空。
2. **海选测试：** 拿海量的、从来没见过的新背景图片，扔进我们已经建好的前 i 关组成的系统中。
3. **收集错题：** 绝大部分新背景会被前几关干掉。但是，凡是能一路骗过前 i 关、最终被误认为是人脸的背景，统统收集起来。这些就是极具欺骗性的“困难样本”(比如长得像人脸的猴子、带有类似五官阴影的墙壁)。
4. **生成新题库：** 把这些骗过系统的困难样本，作为下一关专属的负样本池 N 。

第四步：检查终极目标（外层循环 —— 决定是否继续建关卡）

当前关卡建完后，计算一下目前整个系统（前 i 关连在一起）的整体假阳率 F_i 是多少？（也就是 $f_1 \times f_2 \times \cdots \times f_i$ ）。

- **如果不合格 ($F_i > F_{target}$):** 说明系统整体防线还不够严，距离目标还有差距。回到第二步，开始建造第 $i + 1$ 关。
- **如果合格了 ($F_i \leq F_{target}$):** 大功告成！停止训练，输出最终的级联分类器。

第四步：困难样本挖掘 (Hard Negative Mining) —— $N \leftarrow \emptyset$

当第 i 关终于训练好（既保证了高检测率 d ，又完成了淘汰部分背景的任务 f ），最巧妙的一步来了：

- 清空当前的负样本池 N 。
- 拿成千上万张**纯背景照片**，扔进我们刚建好的前 i 关组成的系统中。
- 绝大部分背景会被前面的关卡淘汰。但是，**凡是能撑过前 i 关还没被淘汰的背景，都是极具欺骗性的“困难样本” (False detections)。**
- 把这些“硬骨头”收集起来，作为下一关（第 $i + 1$ 关）的专属负样本池 N 。

这也就是为什么越往后的关卡特征越多、越复杂：因为它们面对的都是长得极其像人脸的背景，必须用极其严苛的手段才能区分。

1. 假阳率 (False Positive Rate, FPR)

通俗理解： 在所有实际上是负类（没病/背景）的样本中，有多少被模型冤枉成正类了？

计算公式：

$$\text{FPR} = \frac{FP}{FP + TN}$$

(分母 $FP + TN$ 就是所有真实的负样本总数)

2. 假阴率 (False Negative Rate, FNR)

通俗理解： 在所有实际上是正类（有病/人脸）的样本中，有多少被模型漏掉了？

计算公式：

$$\text{FNR} = \frac{FN}{TP + FN}$$

(分母 $TP + FN$ 就是所有真实的正样本总数)



目标检测识别中的几个问题——判别式与生成式

$$\underbrace{\frac{p(\text{zebra} | \text{image})}{p(\text{no zebra} | \text{image})}}_{\text{posterior ratio}} = \underbrace{\frac{p(\text{image} | \text{zebra})}{p(\text{image} | \text{no zebra})}}_{\text{likelihood ratio}} \cdot \underbrace{\frac{p(\text{zebra})}{p(\text{no zebra})}}_{\text{prior ratio}}$$

- 判别式对后验概率建模
- 生成式对似然概率和先验概率建模



目标检测识别中的几个问题——生成式

■ 建模

$p(\text{image} | \text{zebra})$ and $p(\text{image} | \text{no zebra})$



$p(\text{image} \text{zebra})$	$p(\text{image} \text{no zebra})$
Low	Middle
High	Middle $\rightarrow$ Low



基于Bags of features的目标识别

1. 特征提取



基于Bags of features的目标识别

1. 特征提取

2. 学习视觉字典



基于Bags of features的目标识别

1. 特征提取

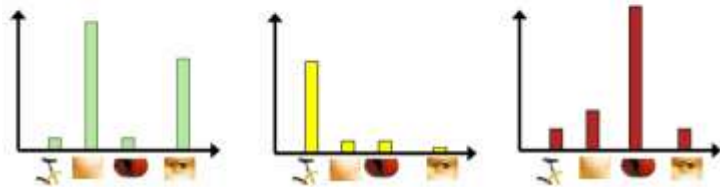
2. 学习视觉字典

3. 利用视觉字典对特征进行量化

基于Bags of features的目标识别

1. 特征提取
2. 学习视觉字典
3. 利用视觉字典对特征进行量化
4. 将图像表达为视觉字典出现的频率

图像表达



特征提取

1. 规则的网格区域



特征提取

1. 规则的网格区域
2. 感兴趣点



规则网格区域的优点是：不容易漏掉东西。

因为它几乎把整张图都扫了一遍。

比如一辆车里面，可能有些区域没有明显的角点，但是仍然有分类信息，例如：车身颜色、车身表面纹理、道路背景、天空区域、阴影区域，这些区域如果用“兴趣点”方法，可能不会被选中。但规则网格会把它们都取出来。所以规则网格区域适合：场景分类 纹理分类 图像整体风格判断 早期目标识别

2.4 这种方法的缺点

缺点也很明显：会取到很多没用的区域。比如天空中一大片纯蓝区域，或者地面中一大片平坦区域，其实信息量不大。

但是规则网格还是会取。所以它会产生大量冗余特征。简单说就是：

优点：覆盖全面，不容易漏；缺点：特征太多，很多没用

感兴趣点：

只在角点、边缘、纹理变化明显等显著位置取特征，信息量高，但可能漏掉一些区域。



特征提取

1. 规则的网格区域
2. 感兴趣点
3. 其他方法（随机采样、基于分割的区域）



特征提取

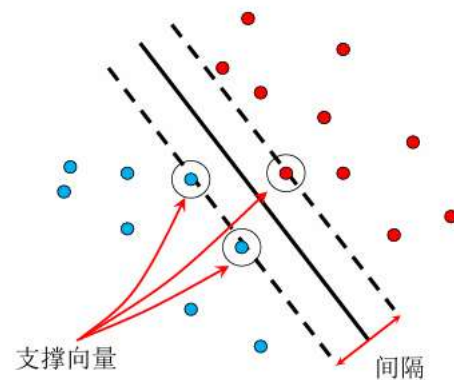


Slide credit: Josef Sivic



目标分类

- 产生式与判别式
 - 最近邻分类器
 - K近邻分类器
 - SVM分类器



45

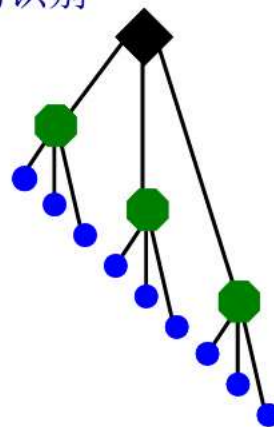
区域归一化的做法
统一中心
统一大小
统一方向
统一尺度

1. 平移归一化：把兴趣点放到 patch 中心
2. 尺度归一化：把不同大小的区域缩放到统一尺寸
3. 旋转归一化：把区域旋转到统一方向
4. 有时还会做亮度/对比度归一化

示例：基于字典树的识别

■ 字典树

■ 训练：完成树结构



[Nister & Stewenius, CVPR'06]

Slide credit: David Nister

55

字典树使用的是**层级聚类**，也可以理解为层级 K-means。

比如每一层分成 3 个子类，分 3 层：

根节点

↓

第一层：分成 3 个大类

↓

第二层：每个大类再分成 3 个小类

↓

第三层：每个小类再分成 3 个更小类



示例：基于字典树的识别

■ 节点权重设置问题

$$w_i = \ln \frac{N}{N_i},$$

■ 距离度量问题

$$\begin{aligned}\|\mathbf{q}-\mathbf{d}\|_p^p &= \sum_i |q_i - d_i|^p \\ &= \sum_{i|d_i=0} |q_i|^p + \sum_{i|q_i=0} |d_i|^p + \sum_{i|q_i \neq 0, d_i \neq 0} |q_i - d_i|^p \\ &= \|\mathbf{q}\|_p^p + \|\mathbf{d}\|_p^p + \sum_{i|q_i \neq 0, d_i \neq 0} (|q_i - d_i|^p - |q_i|^p - |d_i|^p) \\ &= 2 + \sum_{i|q_i \neq 0, d_i \neq 0} (|q_i - d_i|^p - |q_i|^p - |d_i|^p),\end{aligned}$$
$$\|\mathbf{q}-\mathbf{d}\|_2^2 = 2 - 2 \sum_{i|q_i \neq 0, d_i \neq 0} q_i d_i,$$

6.1 公式里的符号是什么意思？

w_i : 第 i 个节点/视觉词的权重

N : 数据库中图像总数

N_i : 包含第 i 个节点/视觉词的图像数量



如果一个视觉词在很多图像里都出现，说明它很常见，区分能力不强。

比如：

普通边缘

天空纹理

灰色背景

地面纹理



这些视觉词到处都有，不能很好地区分图像。

它们的 N_i 很大，所以：

N / N_i 比较小

$\ln(N / N_i)$ 也比较小



权重就低。

7. 距离度量问题

这一页下面写的是：

距离度量问题



也就是说，当每张图片都被表示成一个视觉词向量之后，怎么比较两张图片相似不相似？

假设：

q: 查询图像的视觉词向量 query vector

d: 数据库图像的视觉词向量 database vector



那么可以计算它们之间的距离：

$||q - d||$



距离越小，说明越相似。

7.1 为什么公式里只关心共同非零项？

视觉词向量通常非常稀疏。

比如字典有 100 万个视觉词，但一张图像可能只出现了其中几百个。

所以向量大部分位置都是 0：

$[0, 0, 0, 0.3, 0, 0, 0.8, 0, \dots]$



如果每次都完整计算百万维向量的距离，太慢了。

所以公式把距离拆开，最后希望只计算那些：

q_i 不为 0 且 d_i 也不为 0





基于特征点的目标检测方法示例

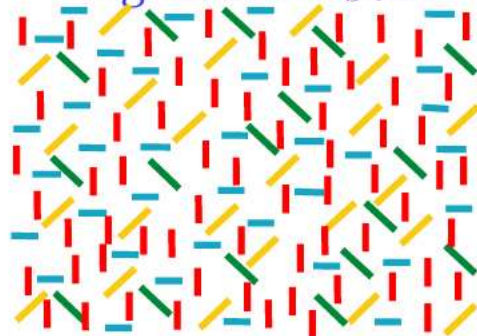
- Bag-of-words框架
- 缺点？
- 如何引入空间信息
- 特征点之间的空间关系
 - 星群模型
 - 空间金字塔模型



基于特征点的目标检测方法示例

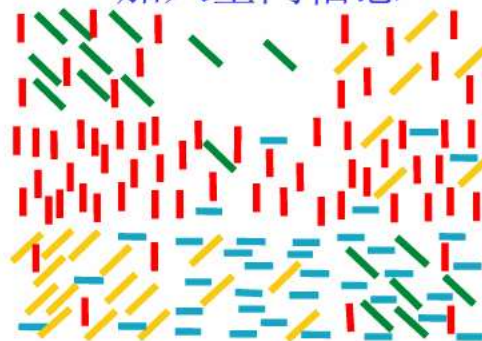


Bag of words 模型



65 17 23 36

加入空间信息



7	8	0	0	0	2	0	0	7	0	4	0
20	0	0	0	11	1	0	2	14	0	3	3
3	0	12	4	0	0	4	16	3	6	0	11

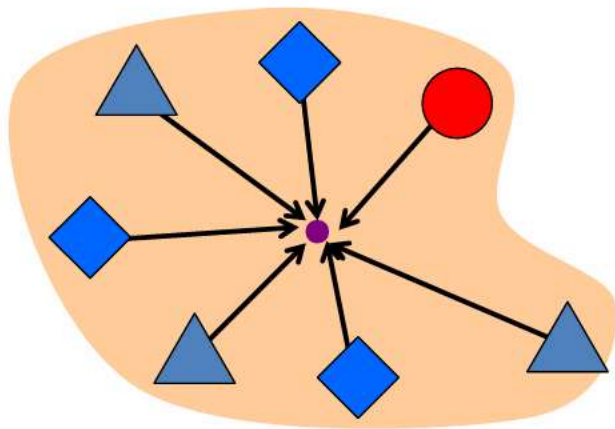
把图像分成几个区域，在每个区域内统计



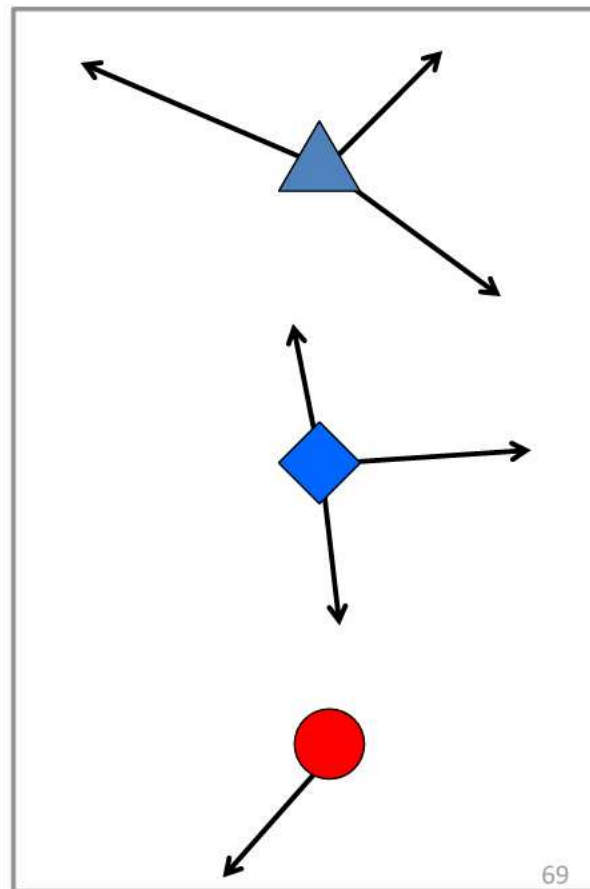
通用霍夫变换——模型构建

对一个类别，用特征，特征相对目标中心的偏移矢量建模

Template



Model

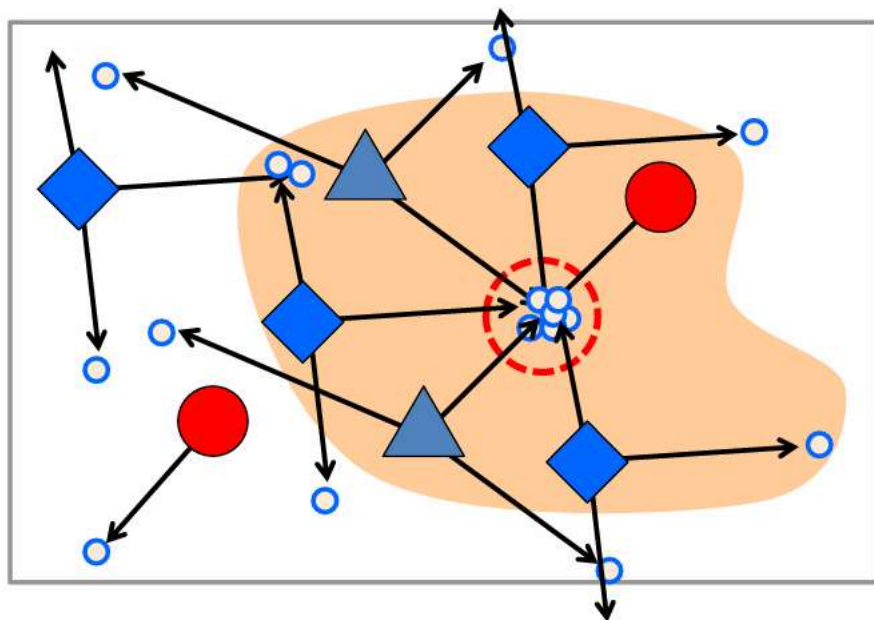




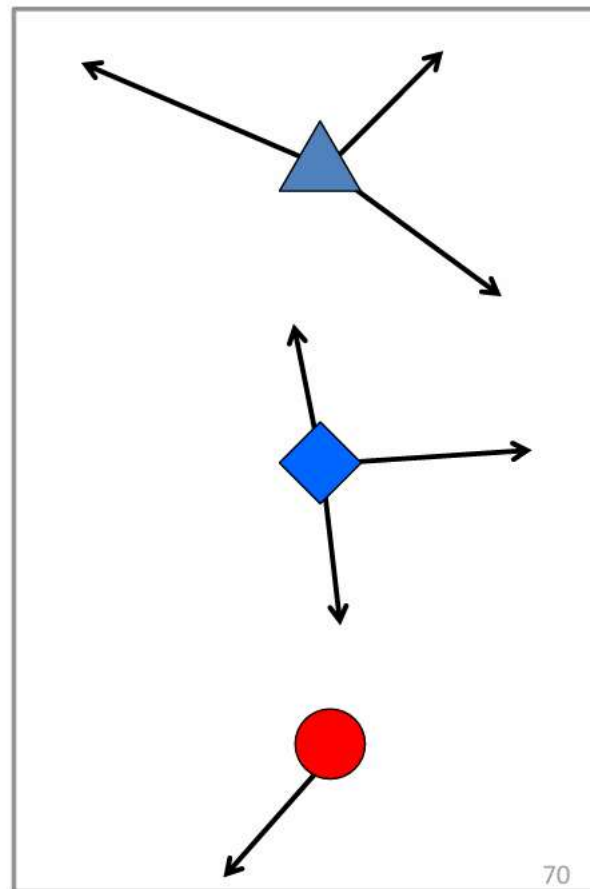
通用霍夫变换——检测

检测时，利用基于特征构建的模型
进行投票

Test image



Model



ISM算法的步骤

一、训练阶段 (Training Procedure)

训练的目的，是建立一个属于该类物体的“视觉词典 (Codebook)”，并记录下每个词汇的空间分布规律。对应图 1 和图 2。

1. 提取局部特征 (Extract Features):

遍历所有训练图像，使用兴趣点检测器 (Interest Point Detector) 找出图像中的关键点，并将关键点周围的图像块 (Patches) 提取出来，计算成特征向量 f_k 。

2. 构建视觉词典 (Build Codebook $\mathcal{C}$):

把所有训练图里提取出来的成千上万个特征向量聚集在一起，使用聚类算法 (Clustering, 设定一个阈值 t) 进行分组。聚类得到的各个中心点，就构成了视觉词典 $\mathcal{C}$ 。你可以把这些词典条目 C_i 理解为该类物体最典型的“标准零件”。

3. 学习空间分布 (Compute Occurrences Occ):

这是 ISM 最关键的一步。再次遍历训练图像：

- 标出当前图像中物体的真实中心坐标 (c_x, c_y) 。
- 提取图像特征 f_k ，并根据相似度 $sim(C_i, f_k) \geq t$ 将它们映射到刚才建立的词典中最接近的条目 C_i 上。
- **记录相对位置：** 对于每一个匹配上的特征，计算它距离物体中心的偏移量（也就是图 2 中那些从图像块指向红点的紫色箭头）： $(c_x - l_x, c_y - l_y)$ 。

- 将这些偏移量存储在对应词典条目的空间概率分布集合 $Occ[i]$ 中。

训练产出物： 最终我们得到了一个模型 $ISM(\mathcal{C}) = (\mathcal{C}, P_c)$ 。它包含了零件库 $\mathcal{C}$ ，以及每个零件相对于物体中心可能出现的位置分布规律 P_c 。

二、识别阶段 (Recognition Approach)

识别的目的，是利用学到的模型，在一张全新的测试图中找出物体的位置。这里使用的是**概率霍夫投票 (Probabilistic Hough Voting)**。对应图 3。

1. 提取测试图特征：

在新的测试图像上应用同样的兴趣点检测器，提取出特征区域 $l_k = (l_x, l_y)$ 和特征向量 f_k 。

2. 匹配视觉词典：

将提取到的特征 f_k 与词典中的条目 C_i 进行比对。如果相似度及格，就认为在图像的 l_k 位置发现了一个“疑似零件”。

3. 概率霍夫投票 (Cast the votes)：

- 既然发现了“疑似零件” C_i ，系统就会去查表：在训练时，零件 C_i 通常距离物体中心有多远？（即取出 $Occ[i]$ 中的相对位置数据 occ ）。
- 利用当前零件在测试图中的实际坐标 (l_x, l_y) ，反向推导（减去偏移量 occ ），**猜测**物体中心 x 应该在哪里。

4. 寻找极大值 (Backprojection of Maxima):

所有的投票都会汇聚到一个 3D 投票空间 (包含 x, y 坐标和尺度 scale)。得票最高、局部最密集的位置 (图 3 右上角的 3D Voting Space 中红蓝点聚集的中心), 就被系统最终认定为**目标的检测中心**。

总结来说: 训练就是收集零件并记住零件到中心的距离; 识别就是拿着新图找零件, 让零件根据记忆去反推中心点, 最后得票最多的中心点就是我们要找的物体。



	<u>Predicted 1</u>	<u>Predicted 0</u>
<u>True 1</u>	true positive	false negative
<u>True 0</u>	false positive	true negative

=

	<u>Predicted 1</u>	<u>Predicted 0</u>
<u>True 1</u>	TP	FN
<u>True 0</u>	FP	TN

	<u>Predicted 1</u>	<u>Predicted 0</u>
<u>True 1</u>	hits	misses
<u>True 0</u>	false alarms	correct rejections

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$



作业

- 分析HOG、颜色直方图、PCA、shapecontext、特征点+BOW这几种表达的平移、旋转（平面内）、尺度、遮挡不变性，并分析原因

	平移	旋转	尺度	遮挡
HOG				
颜色直方图				
PCA				
shapecontext				
特征点+BOW				